Grocermind - Federated Querying over Structured Databases and LLMs

Harsh Rajput
Indraprastha Institute of Information Technology
Delhi, India
harsh22201@iiitd.ac.in

Grishma Bellani
Indraprastha Institute of Information Technology
Delhi, India
grishma22189@iiitd.ac.in

Abstract

Database queries traditionally operate under the closed-world assumption, providing no answers to the questions that require knowledge beyond structured data. Hybrid querying using SQL offers an alternative by integrating relational databases with large language models (LLMs) to answer beyond database questions. In this paper, we present a federated query system that combines structured data from the Blinkit and Zepto grocery platforms with unstructured data processed through an open-source LLM. The system interprets natural language queries and decomposes them into structured SQL sub-queries and unstructured LLM sub-queries for execution across heterogeneous data sources. Furthermore, the system incorporates a user access control mechanism that differentiates between administrative users with full access privileges and customers with restricted data visibility. This ensures that query responses remain accurate, secure, and aligned with user authorization levels in a federated querying environment.

CCS Concepts

• **Information systems → Query languages; Information retrieval; Information integration**.

Keywords

Hybrid Query, Relational Databases and Large Language Models

ACM Reference Format:
Harsh Rajput and Grishma Bellani. 2024. Grocermind - Federated Querying over Structured Databases and LLMs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The relational model and SQL have long been the foundation of modern data management systems. SQL continues to evolve, incorporating new syntactic and functional extensions to meet the increasing demands of large-scale, data-intensive applications. Traditionally, however, SQL queries are evaluated under the *closed-world assumption*, meaning that answers are derived solely from the data stored within the relational database itself. Consequently, classical database systems are unable to address *beyond-database* questions—queries requiring external knowledge not captured in the underlying structured schema. While prior research has explored alternatives such as heuristic reasoning over incomplete data and crowdsourcing-based query answering, these approaches remain limited in scope and reliability.

In parallel, the natural language processing (NLP) community has made substantial progress in *open-domain question answering*, where large language models (LLMs) provide factual answers based on extensive unstructured corpora of the internet. Owing to their pre-training on large knowledge bases, LLMs exhibit strong reasoning capabilities and the ability to supply world knowledge beyond the confines of structured databases. However, beyond-database questions in practical applications often require integration: partial information must be retrieved from relational tables, while the remaining contextual or semantic knowledge must be inferred from unstructured sources. This hybrid nature sets them apart from purely open-domain questions and requires coordinated querying over heterogeneous data modalities.

These challenges are particularly evident in modern grocery delivery ecosystems such as *Blinkit* and *Zepto*, where users frequently need to compare prices, check product availability, and analyze purchase histories across multiple independent platforms. Administrators, on the other hand, require deeper analytical insights—such as category-level trends, district-wise performance metrics, and store-level revenue breakdowns. Since these platforms operate independently and maintain separate relational databases, users cannot easily retrieve integrated information without manual cross-checking. Compounding the challenge, most users lack knowledge of SQL, making it difficult for them to express complex information needs across distributed sources.

While Natural Language Interfaces to Databases allow users to pose queries in natural language and automatically translate them into SQL, traditional NLIDB systems assume a *single centralized database* and therefore fail in distributed, heterogeneous environments typical of real-world e-commerce platforms.

In this paper, we introduce a *federated natural language query system* that integrates structured and unstructured reasoning within a unified architecture. Our system addresses three core challenges:

(1) **Data Integration Across Independent Databases:** We federate structured data from Blinkit and Zepto using classical database federation techniques, enabling cross-platform SQL querying without centralizing or merging the underlying data.
(2) **Answering Beyond-Database Questions via LLMs:** Natural language queries are analyzed and decomposed into (a)

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

SQL sub-queries executed over federated relational sources and (b) unstructured sub-queries processed through an open-source LLM to provide knowledge beyond the database.

(3) **Secure Query Processing Using Role-Based Access Control (RBAC):** The system enforces differentiated privileges for administrators and customers, ensuring that sensitive information remains protected even when queries span multiple platforms and incorporate LLM-generated results.

Our system demonstrates that modern LLM reasoning can be effectively combined with classical database federation to provide a powerful, secure, and intuitive natural language query interface. By enabling both operational lookups and analytical queries across multiple grocery platforms—and answering beyond-database questions—our system offers a unified approach to structured and unstructured query processing while preserving data privacy and access control.

2 Database Construction

To support hybrid querying across heterogeneous data sources, we constructed two independent structured relational databases representing the grocery platforms **Blinkit** and **Zepto**. Although both platforms operate in the same domain, their schemas, attribute names, data conventions, and identifier spaces differ significantly. This section describes the schema design, data generation methodology, and integration challenges involved.

2.1 Overview of Data Sources

The product-level datasets for both platforms were obtained from publicly available resources on Kaggle[1]. These datasets contain detailed item-level information such as product names, categories, price attributes, and descriptions. All additional relational tables including customer records, orders, and order-item details—were synthetically generated to simulate realistic grocery commerce workloads.

To better reflect real-world behavior:

- Approximately **30% of all customers are common** across both Blinkit and Zepto.
- All customers are located in **Mumbai or Delhi**, ensuring overlapping geographic coverage.
- Synthetic orders follow realistic temporal distributions and payment preferences.

2.2 Blinkit Database Schema

The Blinkit schema consists of four tables: `blinkit_products`, `blinkit_customers`, `blinkit_orders`, and `blinkit_order_items`. The product table contains fields such as product name, category, sub-category, pricing attributes, and customer-facing descriptions. Customer profiles include personal information, address fields, and a membership tier:

- **Membership levels:** `bronze`, `silver`, `gold`.
- **Payment methods:** `cod`, `upi`, `card`.
- **Order status:** `pending`, `delivered`, `cancelled`.

[1]https://www.kaggle.com/datasets

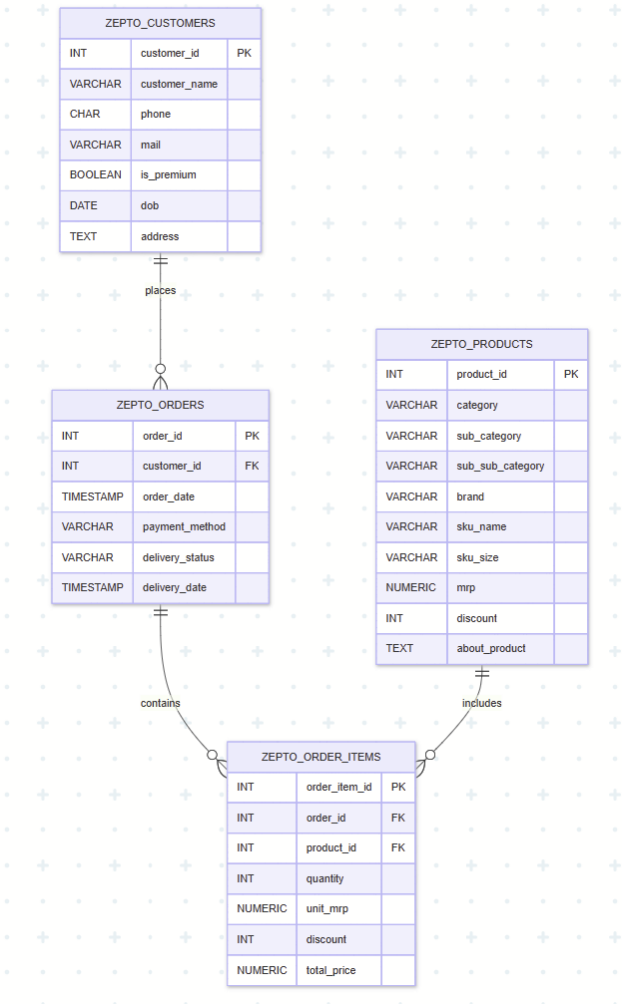


Figure 1: ER Diagram of the Blinkit Database

Orders reference customers and capture delivery timestamps, while order-items record line-level details including product IDs, quantities, and the sale price at the time of purchase.

The Zepto schema also includes four major tables: `zepto_products`, `zepto_customers`, `zepto_orders`, and `zepto_order_items`. While conceptually similar to Blinkit, Zepto adopts different naming conventions and introduces additional attributes such as sub-sub-categories and SKU size.

Key characteristics include:

- **Customer memberships:** represented by a Boolean field `is_premium`.
- **Payment methods:** `upi`, `card`, `cod`.
- **Delivery status:** `pending`, `delivered`, `cancelled`.
- **Order-item pricing:** automatically computed using generated columns incorporating discounts.

The two structured databases capture realistic e-commerce behavior while intentionally preserving independence between Blinkit and Zepto. This independence introduces meaningful challenges

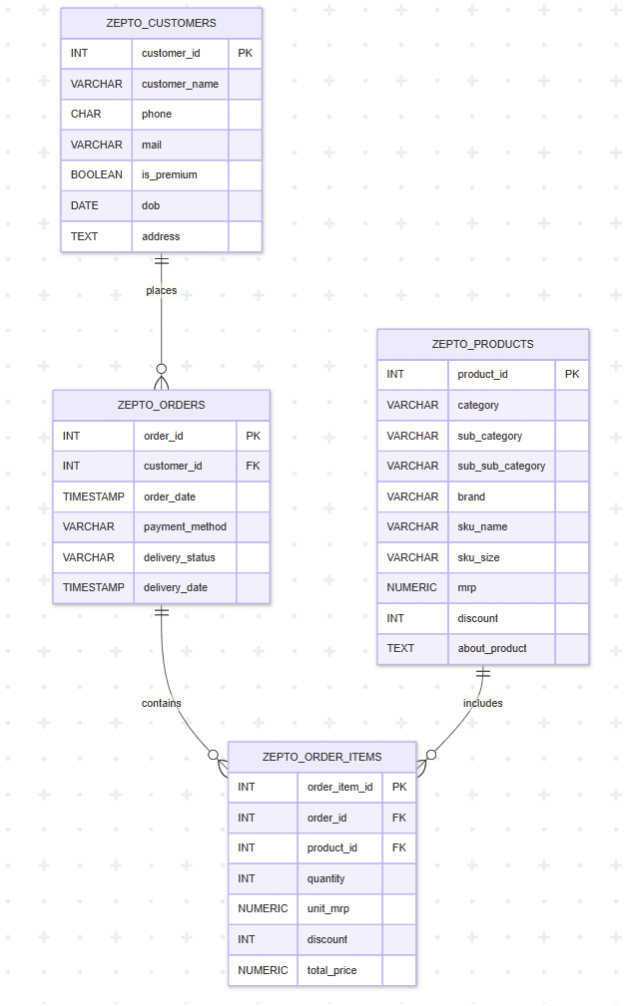


Figure 2: ER Diagram of the Zepto Database

for data integration—particularly for cross-platform customer or product queries—which our hybrid SQL+LLM federated system is designed to address.

3 Global Schema Mapping and Integration Architecture

A major challenge in integrating the Blinkit and Zepto databases arises from the fact that both platforms maintain their data independently. As a result, not only do their identifiers (product IDs, customer IDs, order IDs) differ, but the *schema designs, attribute names, and table semantics also vary significantly.* For example, Blinkit uses attributes such as `product`, `type`, and `market_price`, whereas Zepto uses `sku_name`, `sub_sub_category`, and `mrp`. This structural heterogeneity prevents direct SQL querying across the two systems.

To address these inconsistencies, we construct a unified federated representation using a **Global-As-View (GAV)** approach. In this strategy, the global schema is explicitly defined as views over the underlying autonomous databases. We manually specify attribute-level correspondences for every relevant concept: products, customers, and transactions.

3.1 Unified Product View

We create a global product view that harmonizes attribute names and ensures consistent semantics across platforms. Blinkit's `product` is mapped to a unified `product_name`, while Zepto's `sku_name` is aligned to the same attribute. Categories, brand names, and price information are standardized, including normalization of list price and sale price attributes. A store identifier is incorporated to retain source provenance information.

Since each database uses independent primary keys, we prepend store-specific prefixes ("B_" and "Z_") to construct globally unique product identifiers.

3.2 Unified Customer View

Customer information is also integrated into a global view. For Blinkit, structured address components (house, area, district, city, pincode) are already present, while Zepto stores the full address as a single unstructured field. We standardize Zepto's customer address by splitting it into corresponding components, enabling declarative cross-store comparisons.

Membership information is also normalized: Blinkit uses tiered levels (`bronze`, `silver`, `gold`), whereas Zepto uses a boolean premium flag. We map both representations into a unified membership descriptor.

Customer identifiers are similarly prefixed to guarantee global uniqueness.

3.3 Materialized Transaction View

In contrast to products and customers, transactional data are immutable once an order is completed. Therefore, we materialize the unified transaction table rather than virtualizing it. This materialized global table consolidates: (i) globally-unique transaction identifiers, (ii) order and customer mappings (iii)quantity, sale price, and total price, (iv) payment method and order date, and (v) the originating store.

Materialization significantly improves query performance for analytical workloads because transaction data are among the largest and most frequently aggregated tables.

3.4 Hybrid Federation Architecture

Our design intentionally combines both **virtualization** and **materialization** to create a hybrid architecture suited to the characteristics of the grocery domain:

1) **Virtualization for Product and Customer Views:** These entities change over time—customers may update their profile information, and product catalogues evolve with pricing, and new items. Virtualizing them ensures the global view always reflects the upd information from both stores without requiring constant refreshes.

2) **Materialization for Transaction View:** Transaction records are static once created. Materializing them avoids re-computation across both sources and enables faster cross-store analytics, such

as revenue summaries, customer purchase histories, and store-comparison queries.

This hybrid approach balances consistency, performance, and maintainability. By providing a clean, unified global schema through GAV mappings, our system enables federated natural language querying over two heterogeneous grocery platforms while hiding underlying structural differences from the end user.

4 System Architecture

The overall architecture of Grocermind integrates classical database technologies with modern large language model (LLM)–based reasoning to enable hybrid querying across heterogeneous grocery platforms. The system is designed as a multi-stage pipeline, where each stage transforms the user's natural language query into progressively more structured representations until a final integrated answer is produced. Figure 3 conceptually illustrates the flow of control through the components.

4.1 Overview of the Query Processing Pipeline

The architecture consists of five interconnected layers, each responsible for a critical aspect of federated hybrid querying:

(1) **Natural Language Interface:** Users issue free-form natural language queries through a web-based interface. No knowledge of SQL or schema details is required.
(2) **Query Analyzer:** The Analyzer interprets the semantic intent of the query using the `openai/gpt-oss-120b` model. It classifies the query (structured, hybrid, undefined), extracts entities and conditions, and identifies which parts rely on structured vs. unstructured information.
(3) **Query Decomposer:** Based on the Analyzer's interpretation, the Decomposer generates:
   - an executable SQL sub-query targeting the unified global views (`grocermind_products`, `grocermind_customers`, `grocermind_transactions`), and
   - an LLM reasoning sub-query for information absent from the structured schema.

   This forms the core of hybrid query federation.
(4) **Federated SQL Execution:** Using PostgreSQL FDW connections to the Blinkit and Zepto databases, structured sub-queries are executed over the virtualized/mapped global schema. Transaction data are materialized for performance, while product and customer information remain virtualized.
(5) **LLM Integration and Final Answering:** The structured SQL results and unstructured LLM prompt are passed to the LLMExecutor, which synthesizes a unified user-facing answer in Markdown. This step performs semantic reasoning, contextual inference, and domain-level enrichment beyond database content.

5 User Access Control

A federated querying environment must ensure that data from multiple sources is not only integrated correctly but also accessed securely. Since our system supports both administrative users and regular customers, it is essential to enforce differentiated visibility rules when queries target unified global views or trigger LLM-based sub-queries. To achieve this, The access control layer follows two foundational principles:

- **Least Privilege:** Users are granted only the minimum access required for their tasks.
- **Transparent Enforcement:** Authorization rules are embedded into database views, ensuring that the correct security policies are automatically enforced regardless of how queries are issued.

5.1 Role-Based Access Control Architecture

To support multi-user scenarios, we define two system roles: *Admin* and *Customer*. Using PostgreSQL's roles, schemas, and security-definer views, we construct two layers of logical visibility:

(1) **Global Views** (`grocermind_products`, `grocermind_customers`, `grocermind_transactions`) — accessible depending on user privileges.
(2) **Personalized Views** (`my_profile`, `my_transactions`) — dynamically filtered based on the authenticated customer's identity.

For customer logins, an additional session-level variable stores the customer's phone number, enabling automatic filtering in personalized views without requiring application-side filtering.

5.2 Role-Based Access and Data Visibility

In our federated grocery environment, each user role corresponds to a specific visibility model:

5.2.1 Administrator Role. The `admin` role corresponds to store analysts, database managers, or business intelligence users. Administrators have unrestricted read access across all global views, including:

- complete product listings from Blinkit and Zepto,
- all customer records across both platforms,
- full order and transaction histories.

This unrestricted visibility enables cross-platform analytics, fraud detection, sales forecasting, inventory comparison, and performance evaluation.

5.2.2 Customer Role. The `customer` role corresponds to end users logged in using their 10-digit phone number where the system sets a session parameter (e.g., `app.current_phone`) that captures the phone number (associated with the user). Personalized views reference this variable internally so that they:

- View the **full product catalog** across both Blinkit and Zepto to enable price comparison and browsing.
- Access **only their own profile information** through the `my_profile` view.
- View **only their personal order history and transactions** via the `my_transactions` view.
- **Cannot** view other customers' profiles, transactions, or any global customer/transaction tables.

Thus, even if a malicious customer attempts to query global views, PostgreSQL's permission model prevents access, while the personalized views guarantee correct filtering.

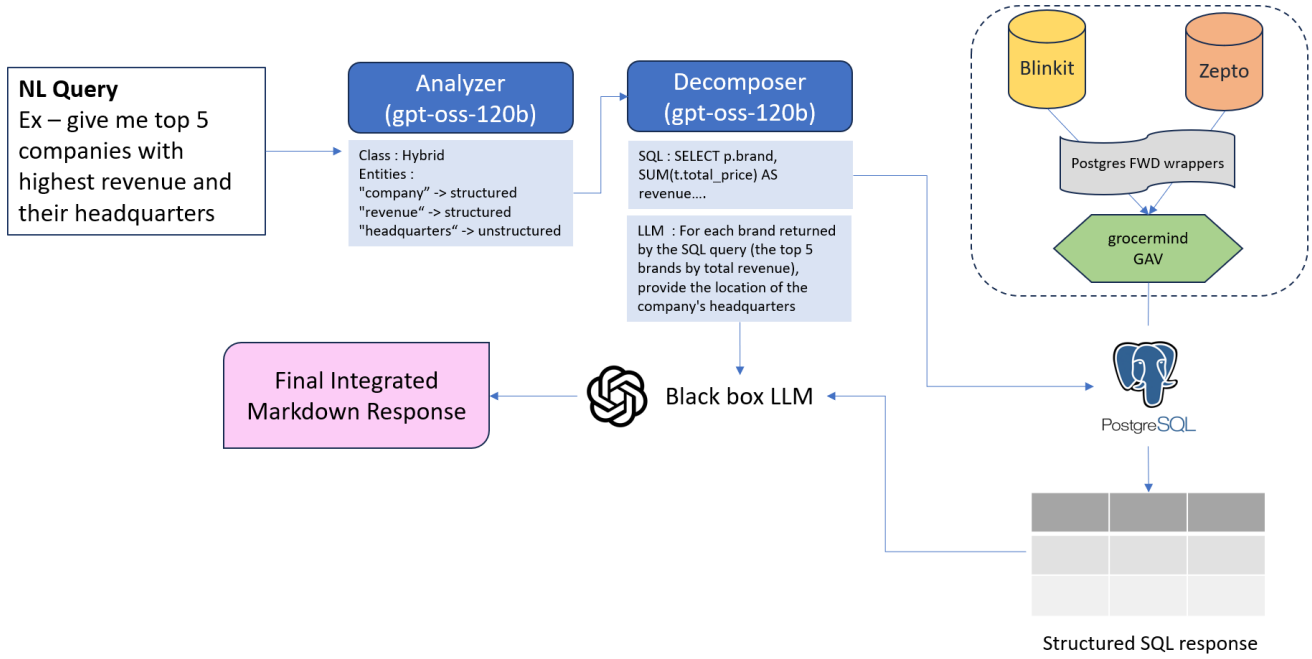


Figure 3: System architecture and hybrid querying pipeline of GROCERMIND.

5.3 Enforcement in the Federated Query Pipeline

Because federation and LLM-based sub-query generation occur *after* authentication and role selection, all query decomposition and execution steps inherently follow the RBAC policies. This ensures:

- SQL sub-queries are executed under the appropriate role,
- LLM-based unstructured sub-queries receive only the non-sensitive information permitted for that user,
- Integrated answers returned to the user never violate data visibility rules.

This integration of RBAC into the federated pipeline prevents accidental or adversarial leakage when answering complex natural language questions.

6 Query Analyzer

The Query Analyzer is the first stage of the federated querying pipeline, responsible for interpreting an incoming natural language query and determining how it should be decomposed across structured SQL execution and LLM-based reasoning. This component ensures that queries over the unified grocery views (`grocermind_products`, `grocermind_transactions`, `grocermind_customers`) are semantically understood before any execution takes place. The Analyzer is implemented using the `openai/gpt-oss-120b`, enabling high-quality semantic parsing.

6.1 Responsibilities and Workflow

Given a user query, the Analyzer produces a structured JSON specification that captures the intent, identified entities, constraints, and a classification of the query type. Specifically, it performs the following tasks:

- **Query Classification:** Determines whether the query is *structured*, *hybrid*, or *undefined*. Structured queries rely purely on SQL fields, while hybrid queries combine SQL-accessible information with external reasoning requiring the LLM.
- **Entity and Condition Extraction:** Identifies all referenced attributes, filters, and relevant metadata present in the natural language query.
- **Structured vs. Unstructured Field Separation:** Distinguishes fields answerable using SQL (e.g., `sale_price`, `order_date`, `city`) from fields requiring world knowledge or inference (e.g., population statistics, carbon footprint of brands).
- **Justification:** Generates a rationale explaining why specific query components are classified as structured or unstructured.

7 Query Decomposer

The Query Decomposer forms the second stage of the federated query processing pipeline. After the Analyzer produces a structured interpretation of the user's natural language query, the Decomposer is responsible for converting this interpretation into executable sub-queries. Specifically, it separates the request into (i) a structured SQL sub-query operating over the unified grocery views and (ii) an unstructured LLM reasoning prompt that addresses information beyond the database. This component is also implemented using the `openai/gpt-oss-120b`.

7.1 Responsibilities and Workflow

The Decomposer consumes the Analyzer's JSON specification and produces a two-part decomposition along with an explanatory rationale. Its key tasks include:

- **SQL Sub-query Generation:** Constructs an executable SQL statement over the unified schema (`grocermind_products`, `grocermind_transactions`, `grocermind_customers`, or user-filtered views such as `my_profile` and `my_transactions`). The SQL is generated using the identified entities, conditions, grouping attributes, and aggregations specified by the Analyzer.
- **Unstructured Query Formulation:** For queries classified as *hybrid*, the Decomposer produces a textual instruction for the LLM to infer beyond database information.
- **Justification:** Provides a clear rationale explaining how the query was partitioned, ensuring transparency and correctness.

8 Structured Data Federation Using PostgreSQL FDW

To enable cross-platform querying over the two independent grocery databases (Blinkit and Zepto), we employ PostgreSQL's native `postgres_fdw` (Foreign Data Wrapper). Both data sources are PostgreSQL instances running on separate hosts, making FDW an ideal lightweight solution for real-time federation without requiring data migration or ETL pipelines.

Foreign Server Configuration. The system establishes two foreign servers—`zepto_server` and `blinkit_server`—each pointing to the corresponding remote PostgreSQL instance. User mappings authenticate the local querying engine to both servers. The schemas from each source are then imported as *foreign tables*, allowing the unified system to access remote relations as if they were local.

Rationale. Using FDW ensures:

- **Live access** to the latest data on both platforms.
- **No duplication**, since the remote tables are accessed directly.
- **Simple integration**, as PostgreSQL provides native SQL pushdown, minimizing network overhead.

Global Unified Views. Once the foreign tables are available locally, we construct unified global views (`grocermind_products`, `grocermind_transactions`, `grocermind_customers`) that combine attributes from both platforms into a consistent schema. These unified views act as the logical integration layer on top of the FDW infrastructure, enabling the Analyzer, Decomposer, and SQL executor to operate without concerning themselves with heterogeneous schemas or remote hosts.

9 Execution and Integration Layer

Once a natural language query has been analyzed and decomposed into structured and unstructured components, the execution pipeline orchestrates their evaluation in a unified manner. This stage consists of two coordinated modules: (1) the **SQL Execution Engine**, responsible for running structured sub-queries over the unified global views, and (2) the **LLM Integration Module**, which answers the unstructured reasoning sub-query and merges its output with the structured results to produce a final response.

9.1 Structured Query Execution

Structured SQL queries are executed through a dedicated Python-based engine implemented using the `psycopg2` PostgreSQL driver. At runtime, the system establishes a database connection to the federated warehouse (`grocermind`) and applies the user's access-control context by setting the appropriate PostgreSQL role (admin or customer). For customer-level sessions, a session variable (`app.current_phone`) is automatically injected so that PostgreSQL views (`my_profile` and `my_transactions`) correctly filter data to the logged-in user.

The SQL Executor returns a Pandas dataframe, ensuring compatibility with downstream LLM processing and enabling clean formatting for user-facing applications, Streamlit.

9.2 LLM-Based Unstructured Reasoning

Unstructured reasoning is carried out by the `LLMExecutor`, which uses the `openai/gpt-oss-120b`. This component receives two inputs:

(1) The tabular result of the SQL execution (truncated to 10 rows for efficiency).
(2) The unstructured LLM sub-query generated by the Decomposer.

The model is then instructed to synthesize a coherent, user-facing answer using both structured and unstructured sources. The LLM is permitted to provide inferred or approximate details when explicitly required by the query, consistent with the needs of hybrid reasoning.

9.3 Integration

The Execution Layer merges SQL-derived facts with LLM-generated insights into a single final response formatted in Markdown. This integrated mechanism enables the system to:

- Produce factually grounded outputs for SQL-accessible information.
- Perform beyond-database reasoning for attributes such as population, product sustainability metrics, or external contextual knowledge.
- Maintain consistency with the federated schema and user access privileges.

By combining deterministic database execution with flexible LLM reasoning, the system provides comprehensive, semantically rich answers that unify structured and unstructured data across both Blinkit and Zepto.

10 Sample Queries

The system supports **Select–Project–Join (SPJ)** queries for structured relational operations and **aggregate** queries such as SUM, AVG, and MAX for analytical reporting. By enabling both transactional retrieval through SPJ operations and analytical summarization through aggregation, the system effectively supports both **OLTP (Online Transaction Processing) and OLAP (Online Analytical Processing)** query patterns. To illustrate its capabilities,

we present representative examples spanning administrative and customer-focused scenarios.

10.1 Sample Queries

- **ADMIN QUERIES**
  - **Query:** What are the top 5 districts of Delhi which generated the highest revenue for Blinkit and also give their approximate population?
    Execution Type: Structured
  - **Query:** Give the product and brand that generated the maximum revenue along with the brand's market cap. (join on product and transaction)
    Execution Type: Hybrid
  - **Query:** Analyze category performance across both stores and recommend which store should focus on which categories.
    Execution Type: Hybrid
  - **Query:** Give me the customers with maximum transactions across both stores.
    Execution Type: Structured
- **CUSTOMER QUERIES**
  - **Query:** Give me all my previous transactions.
    Execution Type: Structured
  - **Query:** Retrieve the customer's previously purchased products and provide better product alternatives.
    Execution Type: Hybrid
  - **Query:** Give me the top 5 cheapest products in the 'hair oil & serum' category across both Blinkit and Zepto and also provide their benefits.
    Execution Type: Hybrid

11 Conclusion

In this work, we presented Grocermind, a federated natural language querying system that unifies structured SQL-based querying with unstructured LLM reasoning across two independent grocery platforms, Blinkit and Zepto. By constructing a hybrid integration architecture—combining virtualized global views for dynamic entities with a materialized transaction warehouse—the system enables seamless cross-platform analytics without centralizing data. The Query Analyzer and Query Decomposer, implemented using the `openai/gpt-oss-120b` model, allow the system to intelligently classify, interpret, and decompose natural language queries into structured and unstructured components. A robust role-based access control layer ensures that administrators and customers receive responses tailored to their authorization levels, preserving data privacy even in a federated environment.

Our execution framework integrates SQL-based results with LLM-generated knowledge, producing coherent final answers that span both structured facts and beyond-database reasoning. This demonstrates that classical database federation techniques and modern large language models can work together to overcome the limitations of closed-world relational systems. The resulting architecture provides an intuitive, secure, and powerful interface for querying heterogeneous data sources, paving the way for next-generation hybrid database-LLM systems.

12 Limitations and Future Work

Although the proposed federated querying system successfully integrates structured relational data with LLM-based reasoning, certain limitations arise from the semantic ambiguity of natural language queries. A key challenge lies in the hierarchical product taxonomy used in our databases, which contain three levels of categorization: `category`, `sub_category`, and `sub_sub_category`. In the current implementation, our global unified view utilizes only the `sub_sub_category` field for product classification. While this simplifies integration across Blinkit and Zepto, it restricts the Analyzer's ability to disambiguate queries that implicitly reference higher-level categories.

For example, consider the query: *"show top 5 selling oil which Zepto customers use for grooming"*. The query does not explicitly identify the filtering attributes or the target category. A direct interpretation might lead the Analyzer to search for items whose `sub_sub_category` contains the substring "oil". However, this would incorrectly include edible oils, even though the user intent is to focus on grooming-related products such as hair oil. Identifying that "grooming" semantically corresponds to broader categories like "hair care" or "personal hygiene" requires deeper contextual understanding and metadata-driven reasoning that the current system does not yet fully support.

As future work, we plan to augment the Analyzer with a feedback-driven semantic refinement mechanism. The enhanced pipeline will leverage database metadata—such as the complete set of categorical values present across all hierarchy levels—and apply semantic similarity search or entity matching to resolve ambiguous product references. When the system detects insufficiently specified attribute–value pairs, it will automatically infer the most plausible filtering conditions, rewrite the analysis instructions, and re-issue the query to the Analyzer for improved decomposition. This adaptive feedback loop is expected to significantly increase accuracy for complex, under-specified, or domain-dependent natural language queries.

Acknowledgments

We would like to express our sincere gratitude to Professor Mukesh Mohania for his invaluable guidance, continuous support, and insightful feedback throughout the duration of this project.

References

[1] Mengyi Liu and Jianqiu Xu. 2025. NLI4DB: A Systematic Review of Natural Language Interfaces for Databases. *arXiv preprint* (2025). arXiv:2503.02435 https://arxiv.org/abs/2503.02435

[2] M. Saeed et al. 2024. Querying Large Language Models with SQL. In *EDBT 2024*. arXiv:arXiv:2304.00472 https://openproceedings.org/2024/conf/edbt/paper-61.pdf Paper on hybrid querying combining LLM and DB.

[3] Matthias Urban and Carsten Binnig. 2023. Towards Multi-Modal DBMSs for Seamless Querying of Texts and Tables. In *preprint (arXiv)*. arXiv:2304.13559 https://arxiv.org/abs/2304.13559

[4] Fuheng Zhao, Divyakant Agrawal, and Amr El Abbadi. 2024. Hybrid Querying Over Relational Databases and Large Language Models. *arXiv preprint* (2024). arXiv:2408.00884 [cs.DB] https://arxiv.org/abs/2408.00884

[5] X. Zhou et al. 2024. DB-GPT: Large Language Model Meets Database. *Proceedings (or arXiv, if applicable)* (2024). https://link.springer.com/article/10.1007/s41019-023-00235-6 Vision paper on LLM-based database framework.